

Guía de Implementación — Microservicios Krypton

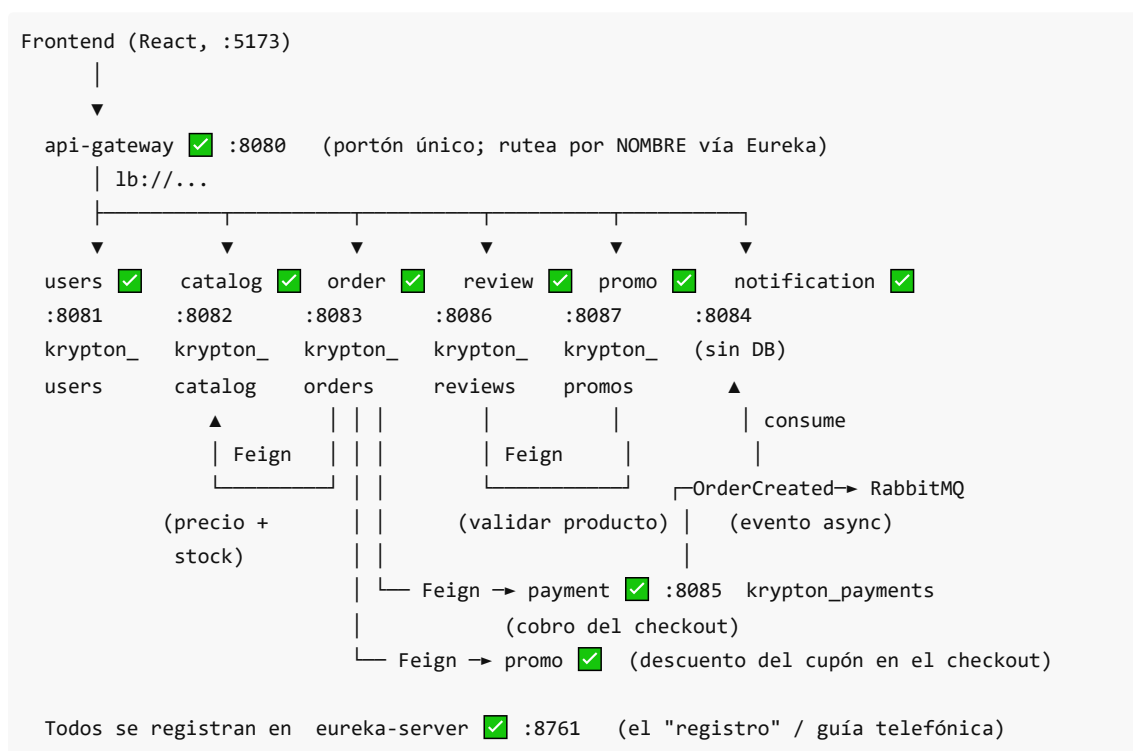
Documento vivo y DIDÁCTICO. No solo muestra el código: lo explica **línea por línea**, con el **porqué** de cada decisión y **dónde conecta** con el resto del sistema. La idea es que entiendas, no que copies. **Versión actual: hasta F11 de 12** (Eureka, users, gateway, catalog, order + checkout distribuido, notification + RabbitMQ, payment + máquina de estados, review, promo, Docker, frontend).

0. Punto de partida y objetivo

Tenemos un **monolito** Spring Boot (backend/) que funciona (411 tests verdes, frontend andando). Lo estamos partiendo en **9 microservicios** para aprender y demostrar los patrones del curso: **Eureka** (descubrimiento), **Feign** (llamadas síncronas entre servicios), **RabbitMQ** (eventos asíncronos), **Docker** y **Kubernetes**.

Regla de oro del proyecto: **no reescribimos lógica**. Reutilizamos el código del monolito, lo repartimos por servicio, y le sumamos la **capa distribuida**. El dominio (productos, usuarios, órdenes) ya está probado; lo que aprendemos acá es cómo esos pedazos **se hablan entre sí** cuando viven en procesos separados.

Arquitectura a la que apuntamos (estado actual marcado en)



Tres ideas que se repiten y conviene tener clavadas desde ya:

- Cada servicio tiene su propia base** (`krypton_users` , `krypton_catalog` , ...). Nadie lee la tabla de otro. Si necesitás un dato ajeno, lo **pedís** (Feign) o lo **escuchás** (RabbitMQ).
- Nadie hardcodea direcciones**. Los servicios se llaman **por nombre** y Eureka traduce nombre → IP:puerto.
- El token JWT es el pasaporte**. Lo emite `users` , y **todos** lo verifican con el mismo secreto, sin volver a preguntarle a `users` .

1. Decisiones de base (el andamiaje)

Monorepo + parent pom + BOM de Spring Cloud

Todos los servicios viven en `services/` como **módulos Maven** de un **parent pom**. El parent fija **una sola vez** la versión de Spring Boot y de Spring Cloud. Este es el *gotcha #1* del stack: **Spring Cloud va atada a una versión específica de Spring Boot**; si las mezclás mal, no arranca.

`services/pom.xml` (lo esencial), comentado:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.5</version>          <!-- 1) hereda gestión de deps de Spring
Boot 3.3.5 -->
</parent>

<groupId>pe.com.krypton</groupId>
<artifactId>krypton-services</artifactId>    <!-- 2) el "paraguas" de todos los
servicios -->
<version>1.0.0</version>
<packaging>pom</packaging>                <!-- 3) NO genera jar: solo orquesta
módulos -->

<properties>
  <java.version>17</java.version>
  <spring-cloud.version>2023.0.3</spring-cloud.version> <!-- 4) versión de Cloud
compatible con Boot 3.3.x -->
</properties>

<modules>                                <!-- 5) los servicios que compila el
parent -->
  <module>eureka-server</module>
  <module>users-service</module>
  <module>api-gateway</module>
  <module>catalog-service</module>
</modules>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>          <!-- 6) el BOM: decide la versión de
CADA starter de cloud -->
    </dependency>
  </dependencies>
</dependencyManagement>
```

Qué hace cada parte numerada:

1. **<parent> Spring Boot**: heredamos el `dependencyManagement` de Boot — por eso en los hijos las deps de Spring van **sin <version>** . 2-3. **packaging=pom** : este módulo no es una app, es un **agregador**. Su único trabajo es compilar a los `<modules>` en orden y compartirles la config.
2. **spring-cloud.version** : una property para no repetir el número.
3. **El BOM** (`<scope>import</scope>`): un "Bill of Materials". Importa una tabla gigante de versiones de TODO el ecosistema Spring Cloud. Gracias a esto, en cada servicio el starter de Eureka/Gateway va **sin <version>** y nunca chocan entre sí.

Por qué importa: mezclar versiones sueltas de Spring Cloud es la causa #1 de errores crípticos al arrancar. El BOM lo elimina de raíz: una sola fuente de verdad para las versiones.

Convenciones que se repiten en TODOS los servicios

- Clase main en `pe.com.krypton.XxxApplication` (en el **paquete base**, para que el escaneo de componentes abarque todo el código carvado, que conserva paquetes `pe.com.krypton.*`).
- `application.yml` con: `server.port` , `spring.application.name` (el nombre en Eureka), `datasource` a **su propio schema**, y `eureka.client.service-url.defaultZone` .
- Una base por servicio (`krypton_users` , `krypton_catalog` , ...) en la misma instancia MySQL.

2. F1 — eureka-server (service discovery)

Concepto: ¿por qué un "registro"?

En un monolito, todo está en el mismo proceso: llamar a otra clase es un método. En microservicios, "el otro" es **otro proceso, en otra IP, en otro puerto**, que además puede **escalar** (3 copias), **caerse** y **revivir** con otra dirección. Si hardcodeás `http://192.168.0.5:8082` , el día que esa instancia cambie, todo se rompe.

Eureka es una guía telefónica viva. Cada servicio, al arrancar, **se anota** ("soy `catalog-service` , estoy en tal IP:puerto"). Los demás, cuando necesitan a `catalog-service` , le **preguntan a Eureka por el nombre** y reciben la dirección actual. Nombres en vez de direcciones.

Código — la app

`eureka-server/pom.xml` lleva **una sola** dependencia de cloud:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>  <!-- el SERVIDOR
de registro -->
</dependency>
```

`EurekaServerApplication.java` :

```
@SpringBootApplication          // 1) app Spring Boot normal (autoconfig + component-scan)
@EnableEurekaServer              // 2) ← LA línea clave: convierte esta app en el registro
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);  // 3) arranca el
contexto y el server web
```

```
}  
}
```

1. **@SpringBootApplication** : el combo de siempre (**@Configuration** + **@EnableAutoConfiguration** + **@ComponentScan**). Sin esto no hay app.
2. **@EnableEurekaServer** : la magia. Levanta el servidor de registro y su panel web. **Es lo único** que diferencia a esta app de un Spring Boot vacío. (Ojo: el SERVER usa **@EnableEurekaServer** ; los CLIENTES usan **@EnableDiscoveryClient** , que viene en F2.)
3. **main** : arranque estándar de Spring Boot.

application.yml — acá está la sutileza:

```
server:  
  port: 8761 # 1) puerto ESTÁNDAR de Eureka (todos lo asumen)  
  
spring:  
  application:  
    name: eureka-server # 2) su nombre  
  
eureka:  
  client:  
    register-with-eureka: false # 3) el server NO se registra a sí mismo  
    fetch-registry: false # 4) ni descarga el registro de otros (él ES el registro)  
  instance:  
    hostname: localhost # 5) cómo se anuncia a sí mismo (en local, localhost)
```

1. **8761** : el puerto canónico de Eureka. Por convención, todos los clientes apuntan acá. 3–4. **register-with-eureka: false / fetch-registry: false** : el detalle que confunde a todos. El servidor de Eureka **también trae un cliente** adentro (porque Eureka puede correr en clúster, replicándose entre varios servers). Como nosotros tenemos **uno solo**, le decimos: "no intentes registrarte ni buscar a otros, vos sos el único registro". Sin estas dos líneas, el server arranca tirando errores de conexión consigo mismo.
2. **hostname: localhost** : en desarrollo, cómo se identifica. En Docker/K8s esto cambia.

Dónde conecta

Es el punto al que **todos** apuntan con `defaultZone: http://localhost:8761/eureka` . Sin él, nadie se encuentra por nombre. Panel: `http://localhost:8761` (lo ves en el navegador con la lista de servicios UP).

Verificado

Arranca en ~6s. El registro arranca **vacío** (`"application":[]`) — correcto, todavía nadie se anotó. En cuanto levantan users y catalog, aparecen ahí como `USERS-SERVICE` y `CATALOG-SERVICE` .

3. JWT a fondo — cómo funciona DE VERDAD

Antes de tocar `users` (que **emite** el token) y `catalog` (que lo **valida**), paremos la pelota y entendamos qué es un JWT por dentro. Si esto no te queda CLARO, la seguridad de todos los servicios va a ser una caja negra. Y nosotros no trabajamos con cajas negras.

El problema que resuelve

HTTP es **sin estado** (stateless): cada request es independiente, el servidor no "recuerda" el anterior. Entonces, ¿cómo sabe el servidor que vos ya te logueaste? Dos enfoques:

- **Sesión clásica (stateful):** el server guarda en memoria/DB "el usuario 42 está logueado" y te manda una cookie con un id de sesión. En CADA request busca esa sesión. Problema en microservicios: si tenés 5 servicios, ¿dónde vive la sesión? Tendrías que compartir un store de sesiones entre todos. Acopla y escala mal.
- **Token autocontenido (stateless): JWT.** El server te da un papel **firmado** que dice quién sos. Vos lo guardás y lo mandás en cada request. El server **no guarda nada**: lee el papel, verifica la firma, y listo. **Esto** es lo que escala a N servicios: cada uno valida el papel por su cuenta.

Anatomía de un JWT: tres partes separadas por puntos

Un JWT es un string que se ve así (cortado para que entre):

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ9LnBlIiwicm9sZSI6IkkFETU10IiwiaWF0Ijo... .
e3dsUMb432QMLrt1QfWVbvq2_V4TG2GTocPATkrmpk...
|----- HEADER -----| |----- PAYLOAD -----|
|----- SIGNATURE -----|
```

Tres partes: **header.payload.signature** . Las dos primeras son **JSON codificado en Base64URL** (NO encriptado — cualquiera lo puede leer). La tercera es la **firma**.

Base64URL ≠ encriptación. Es solo una forma de representar bytes con caracteres seguros para URLs. Si agarrás el header `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ9LnBlIiwicm9sZSI6IkkFETU10IiwiaWF0Ijo...` y lo decodificás (lo hicimos hoy con `base64 -d`), sale:

```
{ "alg": "HS384" }
```

Y el payload decodificado de NUESTRO token real es:

```
{ "sub": "admin@krypton.pe", "role": "ADMIN", "iat": 1782490665, "exp": 1782577065 }
```

- **sub** (subject): a quién pertenece el token → el email.
- **role** : un **claim** (afirmación) nuestro, custom → el rol. Esto es lo que lee `catalog` para decidir si sos ADMIN, **sin tocar la DB**.
- **iat** (issued at) y **exp** (expiration): timestamps Unix. El token vence en `exp` .

🔑 **Punto que la gente no entiende:** el payload **NO está oculto**. Cualquiera que tenga el token puede leer tu email y tu rol. JWT **no da confidencialidad**, da **integridad**: garantiza que nadie **modificó** el contenido. Por eso **NUNCA** metas un password o un dato secreto en el payload.

La firma: por qué NO se puede falsificar

Acá está el corazón. La firma se calcula así:

```
firma = HMAC-SHA384( base64url(header) + "." + base64url(payload) , SECRETO )
```

`HMAC` es una función que toma **dos cosas** — el contenido y una **clave secreta** — y escupe un hash. Propiedades clave:

- **Determinista:** mismo contenido + mismo secreto = misma firma, siempre.
- **Sensible:** si cambiás UN carácter del payload, la firma cambia por completo.

- **Irreversible sin el secreto:** sin conocer el SECRETO, no podés calcular una firma válida.

Entonces, ¿qué pasa si un atacante intercepta tu token y cambia `"role": "ADMIN"` ... perdón, `"role": "CLIENTE"` por `"role": "ADMIN"` ?

1. Modifica el payload → el JSON ahora dice ADMIN.
2. Pero la firma vieja ya **no corresponde** a ese payload nuevo.
3. Para que cuele, necesitaría **recalcular la firma**... y para eso necesita **el SECRETO**.
4. El secreto vive **solo en los servidores**, nunca viaja. → **No puede**. El servidor recalcula la firma del payload recibido, ve que no coincide, y rechaza el token. 401.

Lo verificamos hoy con openssl: tomamos el `header.payload` del token real, calculamos `HMAC-SHA384` con el secreto `dev-only-...`, y la firma nos dio **idéntica** a la del token. Esa es exactamente la cuenta que hace el servidor en cada request.

Por qué nuestro token es HS384 (y no HS256)

Detalle fino y real de NUESTRO código. En `JwtService` la clave se arma así:

```
this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
```

`Keys.hmacShaKeyFor` **elige el algoritmo según el LARGO del secreto:**

Largo del secreto	Algoritmo que elige
≥ 256 bits (32 bytes)	HS256
≥ 384 bits (48 bytes)	HS384
≥ 512 bits (64 bytes)	HS512

Nuestro secreto `dev-only-change-me-krypton-secret-key-1234567890` tiene **exactamente 48 bytes = 384 bits**, así que jwt usa **HS384** (lo confirma el header `{"alg": "HS384"}`). Curiosidad: el javadoc de la clase dice "HS256" — **es impreciso**, el runtime manda. Moraleja didáctica: el algoritmo no lo decide un comentario, lo decide el **largo de la clave**.

Stateless: el secreto compartido es el "contrato"

```
users-service —firma con SECRETO→ TOKEN —verifica con el MISMO SECRETO→ catalog-  
service  
    (emisor)  
    (validador)
```

`users` es el **único** que puede emitir tokens (es dueño de la tabla `users` y conoce los passwords). Pero CUALQUIER servicio que tenga **el mismo secreto** puede **verificar** un token sin preguntarle a `users`. Por eso `catalog` valida en milisegundos, offline, sin llamadas de red. El secreto compartido es **el contrato de seguridad** entre servicios. Cambialo en uno y rompés a todos: esa es justo la prueba de que el sistema funciona.

Con esto en la cabeza, las dos próximas fases (users que EMITE, catalog que VALIDA) se leen solas.

4. F2 — users-service (el emisor del JWT + primer cliente de Eureka)

Concepto

Es el primer servicio que **se registra** en Eureka. Convierte un Spring Boot normal en cliente de Eureka con **3 cosas**: la dependencia, el `application.name` y el `defaultZone`. Carva del monolito la lógica de **usuarios + autenticación**, con su base `krypton_users`. Y es el **único que emite** el JWT.

La app (cliente de Eureka)

```
@SpringBootApplication
@EnableDiscoveryClient           // ← convierte la app en CLIENTE de Eureka (se registra al
                                arrancar)
public class UsersServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(UsersServiceApplication.class, args);
    }
}
```

La **única** diferencia con el eureka-server es la anotación: acá `@EnableDiscoveryClient` (me anoto y busco a otros), allá `@EnableEurekaServer` (yo SOY el registro).

`application.yml` — las "3 cosas" + su base + el JWT:

```
server:
  port: 8081
spring:
  application:
    name: users-service           # 1) nombre con el que lo encuentran
    los demás
  datasource:
    url: jdbc:mysql://localhost:3307/krypton_users # 2) SU base, nadie más la toca
    username: krypton
    password: krypton
  jpa:
    hibernate:
      ddl-auto: validate         # 3) Flyway manda el schema;
    Hibernate solo valida
      open-in-view: false
  flyway:
    enabled: true
app:
  jwt:
    secret: ${JWT_SECRET:dev-only-change-me-krypton-secret-key-1234567890} # 4) el secreto
    (override por env en prod)
    expiration: 86400000        # 5) 24h en milisegundos
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka # 6) dónde está el registro
```

2. `/krypton_users` : su base propia. Aislamiento de datos = cada servicio dueño de lo suyo.

3. **ddl-auto: validate** : Hibernate NO crea ni modifica tablas; solo verifica que el schema (que arma Flyway) matchee las entidades. Si no coincide, no arranca. Disciplina pura.
4. **\${JWT_SECRET:default}** : lee la env var `JWT_SECRET` ; si no existe, usa el default de dev. En prod inyectás un secreto real por entorno. Este MISMO valor tiene que estar en catalog.
5. **expiration: 86400000** : 86.400.000 ms = 24h. Cuánto vive el token.
6. **defaultZone** : la dirección del registro. Las "3 cosas" para ser cliente son los puntos 1 (nombre), 6 (registro) y la dependencia eureka-client del pom.

El login: dónde NACE el token

`AuthController.java` — la puerta de entrada HTTP:

```
@RestController
@RequestMapping("/api/auth")           // 1) todas las rutas cuelgan de /api/auth
public class AuthController {

    private final AuthService authService; // 2) depende de la INTERFAZ (no del impl) →
    testeable

    public AuthController(AuthService authService) { // 3) inyección por constructor
        this.authService = authService;
    }

    @PostMapping("/login")              // 4) POST /api/auth/login
    public AuthResponse login(@Valid @RequestBody LoginRequest request) { // 5) valida y
    deserializa el body
        return authService.login(request); // 6) delega TODA la lógica al service
    }
}
```

2. **Depende de AuthService (interfaz)**, no de la implementación → en los tests mockeás la interfaz. Regla de oro de la arquitectura por capas.
3. **@Valid @RequestBody** : Spring deserializa el JSON a `LoginRequest` y corre las validaciones (email no vacío, etc.). Si fallan → 400 automático, ni entra al método.
4. **El controller NO tiene lógica**: delega. Su trabajo es HTTP (rutas, status, serialización).

`AuthServiceImpl.login(...)` — la lógica real:

```
@Override
@Transactional(readOnly = true)        // 1) solo lee; readOnly optimiza
public AuthResponse login(LoginRequest request) {
    Usuario user = userRepository.findByEmail(request.email()) // 2) busca por
    email

    .orElseThrow(() -> new InvalidCredentialsException("Credenciales inválidas"));
    if (!passwordEncoder.matches(request.password(), user.getPassword())) { // 3) compara
    hash, NUNCA texto plano
        throw new InvalidCredentialsException("Credenciales inválidas");
    }
    if (!user.isActive()) { // 4) usuario dado
    de baja no entra
        throw new InvalidCredentialsException("Credenciales inválidas");
    }
}
```



```

        return new AuthResponse(jwtService.generateToken(user), // 5) ← acá NACE el
token
                                "Bearer", expirationMs / 1000); // 6) tipo +
segundos de vida
    }

```

2. `findByEmail` devuelve `Optional` ; si no existe, `orElseThrow` .
3. `passwordEncoder.matches(plano, hash)` : BCrypt. En la DB el password está **hasheado**; comparás el texto que llegó contra el hash. Nunca se guarda ni compara en plano. 4-. **Mismo error para los 3 casos** (email inexistente / password mal / inactivo): a propósito, no le revelamos a un atacante **qué** emails existen. Detalle de seguridad real.
4. `jwtService.generateToken(user)` : el momento exacto en que se crea el JWT. Le pasamos el usuario entero porque adentro necesita su email (subject) y su rol (claim).

`JwtService.generateToken(...)` — la fábrica del token:

```

public String generateToken(Usuario user) {
    Date now = new Date();
    return Jwts.builder()
        .subject(user.getEmail()) // 1) sub = email → el
"sub" del payload
        .claim("role", user.getRole().name()) // 2) claim custom: el rol
viaja DENTRO del token
        .issuedAt(now) // 3) iat = ahora
        .expiration(new Date(now.getTime() + expirationMs)) // 4) exp = ahora + 24h
        .signWith(key) // 5) FIRMA con el
secreto (HMAC) → la 3ra parte
        .compact(); // 6) arma
"header.payload.signature" y lo serializa
}

```

1. `subject` → el `sub` del payload (el dueño del token).
2. `claim("role", ...)` : metemos el rol COMO DATO en el token. `user.getRole().name()` convierte el enum `Rol.ADMIN` en el String `"ADMIN"` . Esto es lo que después `catalog` lee para autorizar **sin tocar la DB**. ← clave del stateless.
3. `exp` : `now + expirationMs` . Pasado ese instante, cualquier validador lo rechaza.
4. `signWith(key)` : calcula la firma HMAC del `header.payload` con el secreto. Sin especificar algoritmo: `jjwt` lo deduce del largo de la clave (384 bits → HS384, ver §3).
5. `compact()` : pega las 3 partes con puntos y devuelve el String final.

La validación en users: contra la DB (¡distinto a catalog!)

`users` tiene la tabla `users` , así que se da el lujo de **revalidar contra la DB** en cada request. ¿Para qué? Para que una **baja lógica tenga efecto inmediato** aunque el token siga vigente.

`JwtAuthenticationFilter` (`users`) — corre **una vez por request**:

```

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain) {
    String header = request.getHeader("Authorization"); // 1) lee el header
Authorization
}

```

```

        if (header != null && header.startsWith("Bearer ")) // 2) ¿viene "Bearer "
        <token>"?
            && jwtService.isValid(header.substring(7))) { // 3) ¿firma OK y no
            expirado?
                authenticate(header.substring(7), request); // 4) si sí,
            autentica
            }
            chain.doFilter(request, response); // 5) sigue la
            cadena (haya o no auth)
        }

private void authenticate(String token, HttpServletRequest request) {
    UserDetails user =
    userDetailsService.loadUserByUsername(jwtService.extractEmail(token)); // 6) ← LEE LA DB
    if (user.isEnabled()) { // 7) ¿sigue activo?
        (baja inmediata)
        var auth = new UsernamePasswordAuthenticationToken(user, null,
        user.getAuthorities()); // 8) identidad + roles
        auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(auth); // 9) "este request
        está autenticado"
    }
}

```

3. **isValid** : verifica firma + expiración (parsea el token con el secreto; si algo falla, tira excepción que se atrapa y devuelve `false`).
4. **chain.doFilter** **SIEMPRE se ejecuta**: el filtro no rechaza; solo **autentica si puede**. Quien rechaza (401/403) es la capa de autorización, más adelante en la cadena, si no hay identidad.
5. **loadUserByUsername** → **toca la DB**. Esta es **LA diferencia con catalog**. `users` vuelve a buscar el usuario para confirmar que existe y sigue activo.
6. **isEnabled()** (= `active` en la entidad): si lo diste de baja, NO se autentica, aunque su token sea válido. Baja inmediata sin esperar a que venza el token.
7. **SecurityContextHolder** : el "casillero" por-request donde Spring Security guarda quién sos.

`CustomUserDetailsService.loadUserByUsername(...)` — traduce tu `Usuario` al modelo de Spring:

```

public UserDetails loadUserByUsername(String email) {
    Usuario user = userRepository.findByEmail(email)
        .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado: " +
        email));
    return User.builder()
        .username(user.getEmail())
        .password(user.getPassword())
        .authorities(new SimpleGrantedAuthority("ROLE_" + user.getRole().name())) // ←
        prefijo ROLE_ obligatorio
        .disabled(!user.isActive())
        .build();
}

```

El detalle que rompe a todos: `"ROLE_" + ...`. Spring Security, cuando usás `hasRole("ADMIN")`, internamente busca la authority `ROLE_ADMIN`. Si no ponés el prefijo `ROLE_`, `hasRole` nunca matchea y te comés 403 eternos.

El prefijo es **convención obligatoria**.

`SecurityConfig` (users) — define qué es público y qué no:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(Customizer.withDefaults()) // 1)
    habilita CORS (config aparte)
        .csrf(csrf -> csrf.disable()) // 2) sin
    CSRF: somos API stateless
        .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS)) // 3) NUNCA
    crear sesión HTTP
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll() // 4)
    login/registro: público
            .requestMatchers("/api/admin/**").hasRole("ADMIN") // 5)
    gestión: solo ADMIN
            .anyRequest().authenticated() // 6) el
    resto: logueado
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authEntryPoint) // 7) sin
    token → 401 JSON
                .accessDeniedHandler(accessDeniedHandler)) // 8) sin
    permiso → 403 JSON
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class); // 9)
    nuestro filtro ANTES del de Spring
        return http.build();
}
```

2. **CSRF disabled**: CSRF protege formularios con sesión/cookies. Nosotros usamos un token en un header, no cookies de sesión → no aplica.
3. **STATELESS**: la línea filosófica. Spring Security **no crea ni usa `HttpSession`**. Cada request se autentica de cero con su token. Coherente con el JWT. 4–6. **Las reglas se evalúan EN ORDEN**, de la más específica a la más general. `/api/auth/**` público (para poder loguearte sin estar logueado), `/api/admin/**` solo ADMIN, todo lo demás autenticado.
4. **addFilterBefore**: enchufamos nuestro `JwtAuthenticationFilter` **antes** del filtro de login por usuario/password de Spring. Así, cuando llega un request, primero intentamos autenticar por token.

Dónde conecta

Al arrancar, se registra en Eureka como `USERS-SERVICE`. El token que emite acá lo **validan los demás** con el mismo secreto (F4). El gateway lo expone en `/api/auth/**` y `/api/admin/users/**`.

Verificado

`USERS-SERVICE` aparece UP en Eureka. `POST :8080/api/auth/login` con `admin@krypton.pe / Admin123!` devuelve un token Bearer. Y `GET /api/admin/users` con ese token → **200** (valida contra la DB).

5. F3 — api-gateway (portón único)

Concepto

Con varios servicios en puertos distintos, el frontend no debería conocer cada uno (¿y si mañana catalog cambia de puerto?). El **gateway** es **una sola puerta** (:8080 , donde ya apunta el front) que **rutea por nombre** vía Eureka y centraliza el **CORS**. El front sigue hablando con :8080 como si fuera el viejo monolito: **cero cambios en el front**.

Por qué el gateway es REACTIVO (no lleva spring-web)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>          <!-- gateway: stack
REACTIVO (WebFlux) -->
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId> <!-- para resolver
lb://nombre -->
</dependency>
```

Gotcha real: Spring Cloud Gateway corre sobre **WebFlux** (reactivo, no bloqueante). Si por inercia agregás `spring-boot-starter-web` (el servlet clásico), **chocan los dos stacks** y la app no arranca. El gateway NO lleva `-web`. Tiene sentido: un portón que solo reenvía miles de requests rinde mejor sin bloquear un hilo por cada uno.

ApiGatewayApplication.java :

```
@SpringBootApplication
@EnableDiscoveryClient          // ← también es cliente de Eureka: necesita preguntar
"¿dónde está X?"
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
```

application.yml — el corazón del ruteo:

```
server:
  port: 8080                # 1) donde apunta el front → no cambia nada del front
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      default-filters:
        - DedupeResponseHeader=Access-Control-Allow-Origin Access-Control-Allow-Credentials
      RETAIN_UNIQUE # 2)
      globalcors:
        cors-configurations:
          '[/*]':                # 3) CORS para TODAS las rutas
            allowedOrigins: ["http://localhost:5173"] # solo el front de dev
```

```

    allowedMethods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]
    allowedHeaders: [Authorization, Content-Type]
    exposedHeaders: [Content-Disposition]      # para que el front lea nombres de
archivo
    allowCredentials: true
    routes:                                     # 4) la tabla de ruteo
      - id: users-auth
        uri: lb://users-service                 # 5) lb:// = resolver por Eureka +
balancear
        predicates: [ "Path=/api/auth/**" ]     # 6) si el path matchea → mandá acá
      - id: catalog-products
        uri: lb://catalog-service
        predicates: [ "Path=/api/products/**" ]
      - id: catalog-admin-products
        uri: lb://catalog-service
        predicates: [ "Path=/api/admin/products/**" ]
    # ... (categories, uploads, admin-users, etc.)
eureka:
  client:
    service-url:
    defaultZone: http://localhost:8761/eureka   # 7) para resolver los lb://

```

2. **DedupeResponseHeader** : gotcha clásico de CORS detrás de gateway. Tanto el gateway COMO el servicio destino pueden agregar `Access-Control-Allow-Origin` . Resultado: el header llega **duplicado** y el navegador lo rechaza. Este filtro deja **uno solo** (`RETAIN_UNIQUE`).
3. **globalcors** : el CORS se define **una vez acá**, no en cada servicio. El navegador solo habla con `:8080` , así que solo el gateway necesita política CORS.
4. **uri: lb://users-service** : el corazón. **lb://** = *load balancer*. NO es una URL fija: el gateway le pregunta a Eureka "¿dónde está `users-service` ?", obtiene la dirección actual, y si hubiera varias instancias, **reparte** (balancea). Nombre, no IP.
5. **predicates: Path=...** : la condición de match. "Si el path empieza con `/api/auth/` , esta ruta aplica." Se evalúan en orden hasta el primer match.

Gotcha operativo (nos pasó): esta config va **dentro del jar**. Si agregás una ruta, hay que **recompilar y reiniciar** el gateway. Y en Windows, el jar está **bloqueado** mientras corre → primero **parás** el proceso, después `mvn package` , después levantás. Si no, el build falla porque no puede sobrescribir el jar.

Dónde conecta

El front pega a `:8080` → el gateway mira el path → elige la ruta → resuelve el nombre por Eureka → reenvía. El `Authorization: Bearer` que mande el front **viaja tal cual** al servicio destino (lo comprobamos: `GET /api/admin/users` por el gateway con token = 200).

Verificado

`POST :8080/api/auth/login` → token (gateway → users). `GET :8080/api/products` → catálogo (gateway → catalog). Preflight CORS responde con **un solo** `Access-Control-Allow-Origin` .

6. F4 — catalog-service (validar un JWT que NO emitiste)

Concepto (el más importante hasta acá)

catalog **no tiene la tabla** `users` . No puede revalidar contra la DB como hace `users`. Y acá brilla el JWT: **es autocontenido**. catalog solo **verifica la firma** con el secreto compartido y **lee el rol del propio token**. Validación **stateless pura**: sin DB, sin sesión, sin llamar a `users`. Esto es lo que aprendiste en §3, ahora en código.

El JwtService de catalog: recortado a SOLO validar

```
@Service
public class JwtService {
    private final SecretKey key;

    public JwtService(@Value("${app.jwt.secret}") String secret) { // 1) SOLO el
secreto (no expiration)
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8)); // 2) misma
clave que users (mismo secreto)
    }

    public boolean isValid(String token) { // 3) ¿firma
OK y no expirado?
        try { parse(token); return true; }
        catch (JwtException | IllegalArgumentException e) { return false; } // 4)
cualquier falla → inválido
    }

    public String extractEmail(String token) { return
parse(token).getPayload().getSubject(); } // 5) lee el sub
    public String extractRole(String token) { return parse(token).getPayload().get("role",
String.class); } // 6) lee el claim
    private Jws<Claims> parse(String token) {
        return Jwts.parser().verifyWith(key).build().parseSignedClaims(token); // 7)
verifica firma y parsea
    }
}
```

1. **Constructor SOLO con secret** : catalog **no emite** tokens, así que no necesita `expiration` ni la entidad `Usuario` . Es la versión "lectora" del `JwtService` de `users`. Misma derivación de clave → **misma firma** → puede validar lo que `users` firmó. 3-4. **isValid** : intenta parsear; si la firma no matchea o expiró, `parse` tira excepción, la atrapamos y devolvemos `false` . Sin ruido.
2. **extractRole** : lee el claim `"role"` del payload. **Esto reemplaza el lookup a la DB**. El rol viene firmado dentro del token; confiar en él es seguro **porque la firma garantiza que nadie lo tocó** (todo §3 era para poder confiar en esta línea).
3. **verifyWith(key)** : el verificador. Recalcula la firma del `header.payload` con la clave y la compara con la firma del token. Si no coinciden → excepción → inválido.

El filtro de catalog: identidad desde los CLAIMS (sin DB)

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain) {
    String header = request.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
        String token = header.substring(7);
        if (jwtService.isValid(token)) {
            // 1)
            firma + expiración
        }
    }
}
```

```

        var authorities = List.of(new SimpleGrantedAuthority("ROLE_" +
jwtService.extractRole(token))); // 2) rol DEL TOKEN
        var auth = new
UsernamePasswordAuthenticationToken(jwtService.extractEmail(token), null, authorities); //
3)
        auth.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(auth); // 4)
autenticado, sin tocar la DB
    }
}
chain.doFilter(request, response);
}

```

Compará con el filtro de users (\$4): users hace `loadUserByUsername` (DB); catalog construye la identidad **directamente de los claims**.

2. **"ROLE_" + extractRole(token)** : arma la authority desde el rol que viene en el token. Cero consultas. Esta es la esencia del microservicio stateless: **confío en el token firmado**.
3. El **"principal" es el email** (String), no un `Usuario` cargado — catalog ni siquiera tiene esa entidad de usuario.
4. **Autenticado sin DB**. Latencia mínima, sin acople a users. El precio: si das de baja a un user en `users`, su token **sigue siendo válido en catalog hasta que expire** (no hay revalidación). Es el trade-off consciente de stateless. (Se resuelve con tokens cortos o una lista de revocación — fuera de alcance del curso.)

`SecurityConfig` (catalog) — casi igual a users, **pero sin** `PasswordEncoder` ni `AuthenticationManager` (catalog no autentica a nadie, solo verifica tokens):

```

.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll() // 1) ver
catálogo: público
    .requestMatchers(HttpMethod.GET, "/api/categories/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/uploads/**").permitAll() // ver
imágenes: público
    .requestMatchers("/api/admin/**").hasRole("ADMIN") // 2) gestión:
solo ADMIN
    .anyRequest().authenticated())

```

1. **GET de catálogo público**: cualquiera puede VER productos sin loguearse (es un e-commerce). Ojo: solo **GET**; crear/editar/borrar caen en `/api/admin/**`.
2. `/api/admin/**` **solo ADMIN**: y acá el rol sale **del token**, validado statelessly.

`application.yml` — el secreto, que **DEBE** ser idéntico al de users:

```

app:
  jwt:
    secret: ${JWT_SECRET:dev-only-change-me-krypton-secret-key-1234567890} # = al de users
(mismo secreto = misma firma)

```

Si este valor difiere aunque sea **un byte** del de users, catalog rechaza todos los tokens (la firma no matchearía). Lo verificamos hoy comparando los dos secretos byte a byte: idénticos. **Ese es el contrato**.

Adaptación de datos: FK entre servicios

La tabla `stock_movement` tenía una FK `created_by` → `users`. Como `users` vive en **otro servicio con otra base**, no se hace FK cruzada: el campo pasa a ser un simple `Long createdBy` (sin relación JPA) y la migración Flyway crea la columna **sin constraint**. Regla general: **una FK que cruza el límite de un servicio se degrada a un id suelto** (`Long`). La integridad referencial entre servicios se cuida en código/eventos, no con constraints de la DB.

Verificado (con el gotcha que cazamos hoy)

- GET :8080/api/products → 10 productos del seed (público). ✓
- POST :8080/api/admin/products **sin** token → **401** (rechaza). ✓
- POST :8080/api/admin/products **con** token admin → **400** de validación. ✓ ← el 400 (y no 401) prueba que el token **se aceptó**: pasó seguridad y llegó al controller, donde `@Valid` rechazó el body vacío.

🐞 **Gotcha real que diagnosticamos:** GET /api/admin/products con token válido da **401**, no 405, aunque el endpoint solo exista para POST. ¿Por qué? El método no mapeado lanza 405 → Spring hace un **forward interno a /error** → la cadena de seguridad se re-evalúa sobre /error, pero el filtro (un `OncePerRequestFilter`) **no corre en error-dispatch** por default → /error queda sin autenticación → 401. Moraleja: **para probar que un servicio acepta el JWT, usá el método REAL del endpoint** (POST/PUT/DELETE), no GET sobre rutas POST-only.

7. F5 — order-service: el checkout DISTRIBUIDO (Feign + saga)

Concepto (el corazón de la migración)

Hasta acá cada servicio vivía aislado. **F5 es la primera vez que un servicio LLAMA a otro**. El carrito y los pedidos viven en `order-service` (base `krypton_orders`), pero el **stock vive en catalog**. El checkout tiene que descontar stock... que está en otra base, en otro proceso.

Dos patrones nuevos:

- **Feign** — llamar a otro servicio como si fuera un método Java (síncrono: order **espera** la respuesta de catalog).
- **Saga + compensación** — como NO hay un `@Transactional` que abarque dos servicios, el "rollback distribuido" se hace **a mano**: si descontaste stock y después algo falla, **deshacés** llamando a un endpoint de compensación.

Decisión de diseño: identidad por EMAIL

El JWT solo trae `email` + `role` (no el `userId`). Y order **no tiene la tabla users**. Entonces keyeamos carrito y órdenes por `user_email` (el `sub` del token), no por `user_id`. Así order es **autosuficiente** para la identidad: no consulta a users ni necesita cambiar el token. Las FK cruzadas se degradan: `product_id` (Long), `user_email` (String), sin constraint.

Código — el cliente Feign

`client/CatalogClient.java` — una **interfaz**; Feign genera la implementación HTTP:

```
@FeignClient(name = "catalog-service")    // 1) "catalog-service" = el nombre en Eureka
(igual que lb://)
public interface CatalogClient {

    @GetMapping("/api/products/{id}")      // 2) Feign arma el GET HTTP y deserializa la
respuesta
```



```

ProductoResponse getProduct(@PathVariable("id") Long id);

@PostMapping("/api/internal/stock/decrease") // 3) descontar stock (lo construimos en
F4)
void decreaseStock(@RequestBody StockMovementRequest request);

@PostMapping("/api/internal/stock/restore") // 4) restaurar stock = COMPENSACIÓN
void restoreStock(@RequestBody StockMovementRequest request);
}

```

1. **name="catalog-service"** : Feign le pregunta a Eureka "*¿dónde está catalog-service?*" y balancea — exactamente como el `lb://` del gateway. Cero IPs hardcodeadas.
2. **Declarativo**: escribís la firma, Feign hace el HTTP. Es el mismo espíritu que un repository de Spring Data, pero contra otro servicio.

`config/FeignAuthInterceptor.java` — reenvía el JWT (sin esto, las llamadas a `/api/internal/**` de catalog darían **401**):

```

@Component
public class FeignAuthInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        if (RequestContextHolder.getRequestAttributes() instanceof ServletRequestAttributes
attrs) {
            String authorization = attrs.getRequest().getHeader("Authorization"); // 1)
token del request entrante
            if (authorization != null && !authorization.isBlank()) {
                template.header("Authorization", authorization); // 2) lo
pega en la llamada Feign
            }
        }
    }
}

```

Feign corre en el **mismo hilo** del request, así que recuperamos el token del usuario que hace el checkout y lo **propagamos** a catalog. El token viaja servicio→servicio.

Código — la SAGA (`OrdenServiceImpl.confirmarCompra`)

El método clave, anotado (recortado a lo esencial):

```

@Transactional
public OrdenResponse confirmarCompra(String email, CheckoutRequest request) {
    validarDocumento(request); // 1) FACTURA=RUC(11) /
BOLETA=DNI(8) → 422 si no
    Carrito cart = carritoRepository.findByUserEmail(email)
        .orElseThrow(() -> new EmptyCartException("...")); // 2) carrito vacío → 400
    List<ItemCarrito> items = itemCarritoRepository.findByCart(cart);
    // ... if (items.isEmpty()) throw EmptyCartException ...

    // 3) por cada ítem: Feign a catalog → precio VIVO; acumular subtotal y SNAPSHOTear la

```

```

línea
    for (ItemCarrito ci : items) {
        ProductoResponse p = catalogClient.getProduct(ci.getProductId());
        subtotal = subtotal.add(p.price().multiply(BigDecimal.valueOf(ci.getQuantity())));
        ItemOrden linea = new ItemOrden();
        linea.setProductName(p.name()); // ← snapshot del nombre (congelado al comprar)
        linea.setUnitPrice(p.price()); // ← snapshot del precio (congelado al comprar)
        // ...
    }
    // 4) envío (gratis si subtotal ≥ S/300), total, IGV desglosado (el precio YA incluye IGV)

    ordenRepository.save(orden); // 5) guardo la orden PENDIENTE → me da el id
    String reference = "ORDER-" + orden.getId();
    var stockReq = new StockMovementRequest(reference, null, stockItems);

    try {
        catalogClient.decreaseStock(stockReq); // 6) DESCONTAR stock en catalog
    } catch (FeignException ex) {
        if (ex.status() == 422) // catalog dijo "no hay stock"
            throw new InsufficientStockException("..."); // → la TX local hace rollback.
        // NADA que compensar.
        throw ex;
    }

    try { // 7) de acá en más, el stock YA se
        // descontó en catalog
        // guardar las líneas + vaciar el carrito
        return ordenMapper.toResponse(orden, lineas);
    } catch (RuntimeException ex) {
        catalogClient.restoreStock(stockReq); // 8) ← COMPENSACIÓN: deshacer el
        // descuento
        throw ex; // + la TX local hace rollback
    }
}

```

Lo que tenés que entender de memoria:

- **Paso 6 (descontar) viene ANTES de terminar la orden.** Si catalog rechaza por falta de stock (422), nada se commiteó allá, y la transacción local revierte la orden. No hay que compensar.
- **Paso 8 es la COMPENSACIÓN.** Si el descuento YA salió bien (catalog commiteó) pero después algo falla localmente, llamamos a `restoreStock` para **deshacer** lo de catalog. La transacción local no puede tocar la base de catalog → la "deshacés" con otra llamada. **Esa es la saga.**
- **Snapshot (paso 3):** `product_name` y `unit_price` se **congelan** en `order_items`. Por eso ver una orden vieja NO llama a catalog (es histórica). El carrito sí muestra precio vivo (Feign).

TDD de la saga

3 unit tests con Mockito (mockeando Feign y los repos): happy path, **stock insuficiente** (422 →

`InsufficientStockException`, sin compensar) y **compensación** (falla al persistir → se llama `restoreStock`). El

concepto se prueba sin levantar nada.

Dónde conecta

El gateway suma rutas `/api/cart/**` y `/api/orders/**` → `lb://order-service`. order llama a catalog por Feign (resuelto vía Eureka), reenviando el JWT. catalog descuenta stock en SU transacción (lo de F4).

Verificado (end-to-end por el gateway)

Login → POST `/api/cart/items` (el carrito trae nombre/precio **vía Feign**) → POST `/api/orders/checkout` → **201** con la orden; el **stock de catalog bajó** (12→10) y quedó un **SALIDA** en su kardex con `reference=ORDER-{id}`. La saga cruzó dos servicios y funcionó.

8. F6 — notification-service + RabbitMQ (eventos ASÍNCRONOS)

Concepto: síncrono vs asíncrono

F5 fue **síncrono**: order llama a catalog y **espera** la respuesta (los necesita acoplados en el tiempo). F6 es lo opuesto: cuando se confirma una orden, order **publica un evento** y **sigue de largo** — no espera, ni sabe quién lo escucha. notification lo consume **por su cuenta**.

¿Por qué importa? **Desacople total**. Mañana querés mandar también un SMS, actualizar métricas y avisar a logística: sumás 3 consumidores **sin tocar order**. order solo grita "¡se creó una orden!" al aire (a RabbitMQ); quien quiera, que escuche.

```
order —publica OrderCreated→ RabbitMQ (exchange) → cola → notification (consume)
(no espera, no sabe quién hay del otro lado)
```

Código — el evento (idéntico en los DOS lados)

`event/OrderCreatedEvent.java` — el mensaje que viaja como JSON:

```
public record OrderCreatedEvent(Long orderId, String userEmail, BigDecimal total) {}
```

Gotcha clave: este record debe tener el **MISMO nombre completo** (`pe.com.krypton.event.OrderCreatedEvent`) en order Y en notification. El converter JSON pone el tipo en un header `__TypeId__` y el consumidor lo usa para saber a qué clase deserializar. Si los paquetes no coinciden, el consumidor no sabe armar el objeto.

Código — lado PRODUCTOR (order)

`config/RabbitConfig.java` :

```
public static final String EXCHANGE = "krypton.events";
public static final String ROUTING_ORDER_CREATED = "order.created";

@Bean TopicExchange eventsExchange() { return new TopicExchange(EXCHANGE); } // 1) a dónde publica
@Bean Jackson2JsonMessageConverter jsonMessageConverter() { // 2)
    serializa a JSON
}
```

```
return new Jackson2JsonMessageConverter();
}
```

1. **Topic exchange:** order publica acá con una *routing key* (`order.created`). order conoce SOLO el exchange — NO sabe qué colas ni consumidores hay detrás. Ese es el desacople.

`messaging/OrderEventPublisher.java` + el controller publican **después** del checkout:

```
// en OrderEventPublisher:
rabbitTemplate.convertAndSend(EXCHANGE, ROUTING_ORDER_CREATED, event); // publica y vuelve
(fire-and-forget)

// en OrdenController.confirmarCompra:
OrdenResponse orden = ordenService.confirmarCompra(email, request); // 1) la orden YA se
commiteó
eventPublisher.publishOrderCreated(new OrderCreatedEvent(...)); // 2) recién ahí
publica
return orden; // (si el checkout
falla, NO se publica)
```

Publicamos **después** de que la orden se confirmó (post-commit): así no notificamos una compra que terminó revertida.

Código — lado CONSUMIDOR (notification)

`config/RabbitConfig.java` declara **su propia cola** y la **bindea** al exchange:

```
@Bean TopicExchange eventsExchange() { return new TopicExchange("krypton.events"); } //
mismo exchange
@Bean Queue notificationsQueue() { return new Queue("order.notifications", true); } //
1) SU cola (durable)
@Bean Binding orderCreatedBinding(Queue q, TopicExchange e) {
    return BindingBuilder.bind(q).to(e).with("order.created"); //
2) "quiero las order.created"
}
```

`listener/OrderNotificationListener.java` — consume:

```
@RabbitListener(queues = "order.notifications") // 1) escucha SU cola
public void onOrderCreated(OrderCreatedEvent event) { // 2) Spring deserializa el JSON
al record
    log.info("[notification] Orden #{id} de {user} por ${total} → enviando confirmación...",
        event.orderId(), event.userEmail(), event.total()); // 3) acá iría el email
real
}
```

Al arrancar, notification **crea** el exchange + la cola + el binding en RabbitMQ (si no existen). Otro servicio podría bindear OTRA cola al mismo exchange sin que order se entere.

Dónde conecta

RabbitMQ corre en Docker (`docker-compose` , puerto 5672 + consola en 15672). order publica; la cola de notification recibe; el listener procesa. **Nada pasa por el gateway** — es comunicación servicio↔broker, no HTTP de cliente.

Verificado

Un checkout real por el gateway, y en el log de notification aparece: `[notification] Orden #2 de admin@krypton.pe por total S/ 2799.00 → enviando confirmación...` . order publicó y siguió; notification consumió por su lado. **Async demostrado.**

9. F7 — payment-service: pago + máquina de estados

Concepto

Una orden recién confirmada (checkout, F5) nace **PENDIENTE**: existe, descontó stock, pero **todavía no está paga**. F7 agrega el pago. Y acá hay una decisión de diseño que conviene entender bien: **¿dónde vive el endpoint de pago?**

El pago es **parte del ciclo de vida de la orden** (una orden se paga, cambia de estado), así que el endpoint `POST /api/orders/{id}/pay` vive en **order**, no en payment. Pero order **no procesa cobros**: eso es responsabilidad de payment. Entonces order **delega** el cobro a payment-service por **Feign** (otra llamada síncrona, como la de F5 a catalog). Si payment aprueba, order **transiciona** la orden `PENDIENTE → CONFIRMADA` .

El concepto NUEVO de esta fase es la **máquina de estados** (`EstadoOrdenPolicy`): no cualquier cambio de estado es válido. Una orden PENDIENTE puede ir a CONFIRMADA o CANCELADA, pero una CONFIRMADA **no puede volver** a CONFIRMADA (no se paga dos veces) ni saltar a ENTREGADO. Las transiciones válidas se declaran en un **mapa**, y un guardia las valida antes de tocar nada.

Código — la máquina de estados

`policy/EstadoOrdenPolicy.java` — solo valida, **no produce efectos** (responsabilidad única):

```
@Component
public class EstadoOrdenPolicy {

    // El grafo de transiciones VÁLIDAS. Lo que no está acá, está prohibido.
    private static final Map<EstadoOrden, Set<EstadoOrden>> ALLOWED = Map.of(
        EstadoOrden.PENDIENTE, EnumSet.of(EstadoOrden.CONFIRMADA,
EstadoOrden.CANCELADA), // 1)
        EstadoOrden.CONFIRMADA, EnumSet.of(EstadoOrden.ENVIADO, EstadoOrden.CANCELADA),
        EstadoOrden.ENVIADO, EnumSet.of(EstadoOrden.ENTREGADO),
        EstadoOrden.ENTREGADO, EnumSet.noneOf(EstadoOrden.class),
// 2) terminal
        EstadoOrden.CANCELADA, EnumSet.noneOf(EstadoOrden.class));
// 2) terminal

    public void assertCanTransition(EstadoOrden from, EstadoOrden to) {
        if (!ALLOWED.getOrDefault(from, EnumSet.noneOf(EstadoOrden.class)).contains(to)) {
            throw new OrderStatusTransitionException("Transición de estado inválida: " +
from + " -> " + to); // 3) 422
        }
    }
}
```

```
}
}
```

1. **PENDIENTE** → **{CONFIRMADA, CANCELADA}**: pagás (confirma) o la abandonás (cancela). El pago apunta justo a la flecha PENDIENTE → CONFIRMADA.
2. **ENTREGADO y CANCELADA son terminales** (`noneOf` = ningún destino): de ahí no se sale.
3. Si la transición no está en el mapa → `OrderStatusTransitionException` → **422**. Es lo que bloquea pagar dos veces la misma orden.

Código — el pago (`OrdenServiceImpl.pagar`)

```
@Transactional
public OrdenResponse pagar(String email, Long orderId, PaymentRequest request) {
    Orden orden = ordenRepository.findByIdAndUserEmail(orderId, email)
        .orElseThrow(() -> new ResourceNotFoundException("Orden no encontrada: " +
orderId));

    // 1) ¿la orden admite pasar a CONFIRMADA desde su estado actual? (la máquina de
estados)
    estadoOrdenPolicy.assertCanTransition(orden.getStatus(), EstadoOrden.CONFIRMADA);

    // 2) cobrar vía Feign a payment-service (otra llamada SÍNCRONA, como F5 a catalog)
    PaymentResponse pago = paymentClient.charge(
        new ChargeRequest(orden.getId(), orden.getTotal(), request.method().name()));
    if (!"APPROVED".equals(pago.status())) {
        throw new PaymentDeclinedException("El pago fue rechazado"); // 3) 402; la TX
revierte
    }

    // 4) pago aprobado → confirmar la orden y registrar el método de pago
    orden.setStatus(EstadoOrden.CONFIRMADA);
    orden.setPaymentMethod(request.method());
    ordenRepository.save(orden);
    return ordenMapper.toResponse(orden, itemOrdenRepository.findByOrder(orden));
}
```

1. **Primero la máquina**: si la orden ya está CONFIRMADA, `assertCanTransition(CONFIRMADA, CONFIRMADA)` no está en el mapa → 422, y **ni siquiera llamamos a payment** (no cobramos de gusto). El guardia va ANTES del efecto.
2. **paymentClient.charge** : Feign hacia payment, reenviando el JWT (el mismo `FeignAuthInterceptor` de F5). order le pasa el id de la orden, el total y el método.
3. **Rechazado** → `PaymentDeclinedException (402)`: la transacción local revierte, la orden sigue PENDIENTE. (En el demo el cobro siempre se aprueba, pero el camino del rechazo está cableado.)

`payment-service` es chico a propósito: `POST /api/payments/charge` crea un `Pago` en `krypton_payments` con `status = APPROVED` (**simulado** — acá iría la pasarela real Niubiz/Culqi):

```
@Transactional
public PaymentResponse cobrar(ChargeRequest request) {
    Pago pago = new Pago();
    pago.setOrderId(request.orderId()); // FK lógica → order-service (sin constraint
```

```

cruzada)
    pago.setAmount(request.amount());
    pago.setMethod(request.method());
    pago.setStatus(PaymentStatus.APPROVED); // SIMULAMOS: siempre aprueba en el demo
    pago.setCreatedAt(Instant.now());
    pagoRepository.save(pago);
    return new PaymentResponse(pago.getId(), pago.getStatus().name());
}

```

Dónde conecta

order → payment por **Feign** (resuelto por Eureka, JWT reenviado). **No hay ruta nueva en el gateway:**

/api/orders/** ya rutea a order, y POST /api/orders/{id}/pay cuelga de ahí. El endpoint de payment (/api/payments/charge) es **interno** — lo invoca order, nunca el front.

Verificado

- Checkout → la orden nace **PENDIENTE**. ✓
- POST /api/orders/{id}/pay con { "method": "YAPE" } → **200 CONFIRMADA** + una fila en krypton_payments (status APPROVED). ✓
- Re-pagar **la misma** orden → **422** ← la máquina bloquea CONFIRMADA → CONFIRMADA: no se paga dos veces. Y como el guardia corre primero, payment ni se entera.

10. F8 — review-service: reseñas de productos

Concepto

Un servicio de dominio **nuevo**: reseñas (rating 1–5 + comentario) por producto. Lo importante didácticamente es que **REUSA todo lo que ya viste**: su propia base (krypton_reviews), JWT stateless con el secreto compartido, registro en Eureka, ruta en el gateway. Es el "pattern-repeat" de un microservicio de dominio: ya sabés armarlo.

Lo nuevo, dos cosas chicas: **agregación** (calcular el promedio de estrellas con una query) y **validación cruzada por Feign** (antes de aceptar una reseña, preguntarle a catalog si el producto existe).

Ojo de ruteo (importante): las reseñas van bajo /api/reviews , NO bajo /api/products/{id}/reviews . ¿Por qué? Porque /api/products/** **ya rutea a catalog** en el gateway. Si colgáramos las reseñas de ahí, el gateway las mandaría a catalog, que no sabe nada de reseñas. El producto viaja como **query param** (? productId=), no en el path.

Código — el repository (la agregación)

```

public interface ResenaRepository extends JpaRepository<Resena, Long> {

    List<Resena> findByProductIdOrderByCreatedAtDesc(Long productId); // listado
del producto

    boolean existsByUserEmailAndProductId(String userEmail, Long productId); // ← un
usuario reseña 1 vez

    @Query("SELECT AVG(r.rating) FROM Resena r WHERE r.productId = :productId") // ← la
AGREGACIÓN

```

```

    Double promedioByProductId(@Param("productId") Long productId);           // null si
no hay reseñas
}

```

- `promedioByProductId` : `AVG(r.rating)` lo calcula la **base**, no Java. Devuelve `Double` (no primitivo) a **propósito**: si el producto no tiene reseñas, `AVG` es `NULL`, y el service lo traduce a `0.0`.
- `existsByUserEmailAndProductId` : respalda en código el `UNIQUE(user_email, product_id)` de la tabla — un usuario, una reseña por producto.

Código — el service (validación cruzada por Feign)

```

@Transactional
public ResenaResponse crear(String email, CreateResenaRequest request) {
    // 1) el producto debe existir en catalog → Feign. Si catalog responde 404, el handler
    lo propaga.
    catalogClient.getProduct(request.productId());

    // 2) un usuario reseña un producto UNA sola vez
    if (resenaRepository.existsByUserEmailAndProductId(email, request.productId())) {
        throw new DuplicateReviewException("Ya reseñaste este producto");    // → 409
    }

    // 3) crear y persistir (la identidad es el email del JWT)
    Resena resena = new Resena();
    resena.setProductId(request.productId());
    resena.setUserEmail(email);
    resena.setRating(request.rating());
    resena.setComment(request.comment());
    resena.setCreatedAt(Instant.now());
    return resenaMapper.toResponse(resenaRepository.save(resena));
}

```

1. `catalogClient.getProduct` : review **no tiene la tabla products**, así que pregunta. Si el id no existe, catalog tira 404 y la reseña ni se intenta. Misma idea que F5: dato ajeno → lo pedís.
2. **Duplicado → 409** (`DuplicateReviewException`) : el código chequea antes de chocar con el `UNIQUE`.

Endpoints (`ResenaController`) : `GET /api/reviews?productId=...` (**público** — cualquiera ve las reseñas) y `POST /api/reviews` (**autenticado** — para opinar hay que estar logueado; la identidad sale de `authentication.getName()`).

Dónde conecta

review → catalog por **Feign** (validar producto). Gateway: `/api/reviews/**` → `lb://review-service` . review se registra en Eureka como `REVIEW-SERVICE` y valida el JWT statelessly (igual que catalog).

Verificado

- `POST /api/reviews` (logueado) → **201** con la reseña. 
- `GET /api/reviews?productId=1` → **promedio + total + listado** (el `AVG` sale de la query). 
- Reseñar dos veces el mismo producto → **409**. 
- Reseñar un producto inexistente → **404** (lo dice catalog, vía Feign). 

11. F9 — promo-service: cupones de descuento

Concepto

Otro servicio de dominio (pattern-repeat puro): **cupones de descuento**. Dos tipos: **PORCENTAJE** (ej. 10% off) o **MONTO** (ej. S/50 fijos). Tiene un CRUD de **admin** (crear/listar cupones) y un endpoint para **aplicar** uno sobre un monto. La integración real con el checkout llega en F11; acá construimos el servicio y su cálculo.

Código — el cálculo del descuento

```
public enum DescuentoTipo { PORCENTAJE, MONTO } // persistido como STRING (nunca ORDINAL)
```

```
@Transactional(readOnly = true)
public DescuentoResponse aplicar(AplicarPromoRequest req) {
    Promo p = promoRepository.findByCode(req.code())
        .orElseThrow(() -> new PromoInvalidaException("Código inválido")); // 1) no
    existe → 422
    if (!p.isActive()) {
        throw new PromoInvalidaException("Código inactivo"); // 1)
    }
    desactivado → 422
    BigDecimal discount;
    if (p.getType() == DescuentoTipo.PORCENTAJE) {
        discount = req.amount().multiply(p.getValue())
            .divide(CIEN, 2, RoundingMode.HALF_UP); // 2)
    } else {
        discount = p.getValue().min(req.amount()); // 3)
    }
    amount * value / 100
    MONTO: nunca más que el total
    BigDecimal finalAmount = req.amount().subtract(discount);
    return new DescuentoResponse(p.getCode(), discount, finalAmount);
}
```

1. **Inválido/inactivo** → **PromoInvalidaException (422)**: no existe el código, o existe pero está apagado. El cliente recibe un error claro, no un descuento fantasma.
2. **PORCENTAJE**: $\text{amount} * \text{value} / 100$, redondeado HALF_UP a 2 decimales (plata, siempre 2 decimales). Un 10% sobre 100 → 10.
3. **MONTO**: $\min(\text{value}, \text{amount})$ — un cupón de S/50 sobre una compra de S/30 descuenta 30, no 50 (no regalamos plata). Detalle de negocio fácil de olvidar.

Endpoints: POST /api/admin/promos y GET /api/admin/promos (**solo ADMIN**, caen bajo /api/admin/** → `hasRole("ADMIN")`); POST /api/promos/apply (autenticado). Crear con un código repetido → **409** (`DuplicatePromoCodeException`).

Dónde conecta

Gateway: /api/admin/promos/** y /api/promos/** → lb://promo-service . promo se registra en Eureka y valida el JWT statelessly. **La integración real con el checkout (que order le pida el descuento) llega en F11** — por ahora promo vive solo.

Verificado

- `POST /api/admin/promos` (ADMIN) → **201** con el cupón. ✓
- `POST /api/promos/apply` 10% sobre 100 → `discount 10` / `finalAmount 90`. ✓
- Cupón inexistente o inactivo → **422**. ✓
- Crear un código repetido → **409**. ✓

12. F10 — Dockerizar todo

Concepto

Hasta acá levantábamos cada servicio a mano (`java -jar ...` , una terminal por servicio). F10 **empaqueta cada servicio en una imagen** y levanta TODO con **un solo** `docker compose up` .

El desafío central: **dentro de Docker, localhost no sirve**. Cuando order corría como jar y le pegaba a `http://localhost:8761` , "localhost" era tu máquina. Dentro de un contenedor, `localhost` es **el propio contenedor** — no hay nadie ahí. En la red de compose, los servicios se hablan por **nombre de contenedor** (`eureka-server` , `krypton-db` , `rabbitmq`). Hay que parametrizar las URLs.

Código — parametrizar por env (sin romper el modo jar local)

La clave es usar `${VAR:default}` con el default en `localhost` : **así el mismo jar sirve para los dos mundos** — local (sin env, usa el default) y Docker (con env, usa el nombre de contenedor):

```
# en CADA servicio (application.yml):
eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka} # local: localhost; Docker:
eureka-server
  instance:
    prefer-ip-address: ${EUREKA_PREFER_IP:false} # ← clave en Docker (ver
abajo)
spring:
  datasource:
    url: jdbc:mysql://${DB_HOST:localhost}:${DB_PORT:3307}/krypton_orders # local :3307;
Docker krypton-db:3306
  rabbitmq:
    host: ${RABBITMQ_HOST:localhost} # local: localhost; Docker:
rabbitmq
```

El detalle fino es `prefer-ip-address` . Por default Eureka registra a cada servicio por su **hostname**; en Docker ese hostname es el id del contenedor, que los demás no saben resolver. Con `EUREKA_PREFER_IP=true` , cada servicio se registra por **su IP en la red de compose**, que sí es alcanzable. En local lo dejamos en `false` (localhost anda).

Un Dockerfile por servicio (mínimo — solo empaqueta el jar ya construido):

```
FROM eclipse-temurin:21-jre # JRE liviano, sin Maven adentro
WORKDIR /app
COPY target/order-service-1.0.0.jar app.jar
```

EXPOSE 8083

ENTRYPOINT ["java", "-jar", "app.jar"]

`docker-compose.full.yml` ata todo. En Docker se inyectan los nombres de contenedor:

```
order-service:
  build: ./services/order-service
  environment:
    EUREKA_URL: http://eureka-server:8761/eureka # nombre del contenedor, no localhost
    EUREKA_PREFER_IP: "true" # registrarse por IP de la red de
  compose
    DB_HOST: krypton-db
    DB_PORT: "3306" # puerto INTERNO de MySQL (no el 3307
  del host)
    RABBITMQ_HOST: rabbitmq
  depends_on:
    krypton-db: { condition: service_healthy } # esperar a que MySQL esté SANO
    rabbitmq: { condition: service_healthy }
    eureka-server: { condition: service_started }
```

Dos cosas más que importan:

- **DB_PORT: 3306** (no 3307): adentro de la red, MySQL escucha en su puerto **interno** 3306. El 3307 era el mapeo al host para que no chocara con un MySQL nativo — irrelevante entre contenedores.
- **Un init SQL** (`docker/mysql-init/01-init.sql` , montado en `/docker-entrypoint-initdb.d`) crea las **6 bases por servicio** (`krypton_users` , `krypton_catalog` , ..., `krypton_promos`) antes de que arranque cualquier Flyway. Las **tablas** las sigue creando el Flyway de cada servicio; el init solo deja las bases vacías.
- **depends_on con condition** : DB y rabbit con `service_healthy` (esperan al healthcheck, no solo a que el proceso exista); eureka con `service_started` . Sin esto, los servicios arrancan antes que su base y mueren.

Gotcha real (Docker Desktop en Windows)

`rabbitmq:3-management` **fallaba al arrancar** con un `eaccess` leyendo `/var/lib/rabbitmq/.erlang.cookie` (permisos del volumen en Docker Desktop Windows). El fix: darle un **volumen propio** (`krypton_rabbitdata_full`) y un **hostname estable** (`hostname: krypton-rabbit`) para que el cookie y el estado de Mnesia sean consistentes entre reinicios.

Y en Windows, antes de `mvn package` : **parar los jars locales** que estén corriendo — bloquean el `.jar` y el build no puede sobrescribirlo (mismo gotcha del gateway en F3).

Verificado

`docker compose -f docker-compose.full.yml up -d --build` levanta **11 contenedores** (MySQL, RabbitMQ, Eureka, gateway + los 7 de negocio). El checkout completo + el pago + el evento RabbitMQ a notification funcionan **entre contenedores**, todo entrando por el gateway en `:8080` . La misma demo de antes, ahora 100% containerizada.

13. F11 — Frontend: reseñas + cupón (con integración real)

Concepto

La última fase de features: conectar el **React** a los servicios nuevos. Pero hay una asimetría importante que conviene tener clara:

- **Reseñas = puro frontend.** El backend (F8) ya está completo. El front solo consume `/api/reviews` : lista, muestra el promedio, deja opinar. No toca el backend.
- **Cupón = NO es solo frontend.** Para que el descuento sea **REAL** (que baje el total que se cobra y se guarda), no alcanza con mostrarlo en pantalla: **order tiene que aplicarlo en el checkout**, llamando a promo por Feign. Si el descuento viviera solo en el front, cualquiera lo falsificaría. **La plata la decide el backend.**

Código — reseñas (lado front)

El patrón de siempre: un `*.api.ts` por feature + un componente que lo consume.

```
// features/catalog/reviews.api.ts
export async function getByProduct(productId: number): Promise<ReviewListResponse> {
  const { data } = await api.get<ReviewListResponse>('/api/reviews', { params: { productId } }); // ?productId=
  return data;
}
export async function create(req: CreateReviewRequest): Promise<ReviewResponse> {
  const { data } = await api.post<ReviewResponse>('/api/reviews', req); // el token lo adjunta el interceptor
  return data;
}
```

El componente `ProductReviews` (en la página de producto) usa `getByProduct` para pintar promedio + lista, y `create` para enviar una reseña nueva. Cero lógica de negocio en el front: pega al gateway y muestra.

Código — cupón (lo IMPORTANTE está en el backend)

`CheckoutRequest` suma un campo opcional `couponCode` :

```
public record CheckoutRequest(
    @NotNull TipoDocumento documentoType,
    @NotBlank @Size(max = 150) String customerName,
    @NotBlank @Pattern(regexp = "\\d{8}|\\d{11}") String customerDoc,
    String couponCode) {} // ← NUEVO: el cupón (opcional)
```

Y `confirmarCompra` (el checkout de F5) ahora pide el descuento a promo por **Feign** y lo mete en el total:

```
// dentro de confirmarCompra, ya con el subtotal calculado:
BigDecimal discount = aplicarCupon(request.couponCode(), subtotal); // 1) Feign a promo (0 si no hay)
BigDecimal shipping = subtotal.compareTo(FREE_SHIPPING_THRESHOLD) >= 0
    ? BigDecimal.ZERO : SHIPPING_COST;
BigDecimal total = subtotal.subtract(discount).add(shipping); // 2) total = subtotal - descuento + envío
// ... orden.setDiscount(discount); ← se persiste (columna nueva, Flyway V2)

private BigDecimal aplicarCupon(String couponCode, BigDecimal subtotal) {
```

```

    if (couponCode == null || couponCode.isBlank()) return BigDecimal.ZERO; // sin cupón → 0
    try {
        return promoClient.applyPromo(new ApplyPromoRequest(couponCode, subtotal)).discount();
    } catch (FeignException ex) {
        if (ex.status() == 422 || ex.status() == 404)
            throw new InvalidCouponException("Cupón inválido: " + couponCode); // 3) → 422
        throw ex;
    }
}

```

1. **aplicarCupon** : si no vino cupón, descuento 0 (no llama a promo). Si vino, **Feign a promo** con el subtotal; promo devuelve el descuento **autoritativo** (el del front era solo preview).
2. **total = subtotal - discount + envío** : el descuento entra en la cuenta real que se cobra y se guarda en la orden.
3. **Cupón inválido** → **InvalidCouponException (422)**: order traduce el 422/404 de promo a un error claro y la transacción del checkout revierte.

La columna `discount` se agrega con una migración **Flyway V2** (nunca se edita la V1 ya aplicada):

```

-- V2__add_order_discount.sql
ALTER TABLE orders ADD COLUMN discount DECIMAL(12,2) NOT NULL DEFAULT 0.00 AFTER subtotal;

```

Del lado front, el modelo `promo.ts` deja explícito que **el descuento real lo calcula el BACKEND**; el front manda `couponCode` en el checkout y muestra el descuento que vuelve.

Dónde conecta

- **front** → **gateway**: reseñas (`/api/reviews`) y preview del cupón (`/api/promos/apply`).
- **order** → **promo por Feign EN el checkout**: el descuento **autoritativo**, el que de verdad baja el total. Esta es la diferencia clave: el cupón no es cosmético, atraviesa el backend.
- **Flyway V2** agrega la columna `discount` a `orders` .

Verificado

- Checkout con cupón 10% sobre un subtotal de **S/2799** → `discount 279.90` , `total 2519.10` (+ envío gratis por superar S/300). 
- Cupón inválido/inactivo → **422** y el checkout revierte. 
- Sin cupón → `discount 0` , total normal. 

14. Cómo conecta TODO (hasta la F11)

1. El front (o curl) le pega al GATEWAY en :8080 con `Authorization: Bearer <token>`.
2. El gateway elige el servicio por PATH y lo resuelve por NOMBRE vía Eureka (lb://).
3. El servicio valida el JWT: users contra su DB; los demás STATELESS (firma + role del token).
4. **SÍNCRONO** (Feign), varias llamadas servicio→servicio que ESPERAN respuesta:
 - checkout: order → catalog (precio + descontar stock) → promo (descuento del cupón). Si falla a la mitad → saga: order COMPENSA (restaura stock).
 - pago: order → payment (cobro); si aprueba, la MÁQUINA DE ESTADOS confirma la orden.

```
- reseña: review → catalog (validar que el producto existe).
5. ASÍNCRONO (RabbitMQ): al confirmar, order PUBLICA un evento y sigue; notification lo consume aparte.
6. Cada servicio trabaja contra SU PROPIA base (krypton_users / _catalog / _orders / _payments / _reviews / _promos).
7. TODO esto puede correr containerizado (docker-compose.full.yml): 11 contenedores, una sola puerta (:8080).
```

Los "pegamentos" del sistema, ahora completos:

- **Eureka** (nombres ↔ direcciones) — nadie hardcodea hosts.
- **El gateway** (una puerta, ruta por nombre, CORS).
- **El secreto JWT compartido** (el contrato de seguridad: users firma, todos verifican).
- **Feign** (llamada **síncrona** servicio→servicio, resuelta por Eureka) + **saga/compensación**. Lo usan order→catalog, order→payment, order→promo y review→catalog.
- **Máquina de estados** (EstadoOrdenPolicy): qué transiciones de la orden son válidas — el pago la dispara PENDIENTE → CONFIRMADA.
- **RabbitMQ** (eventos **asíncronos**: publicar sin saber quién escucha).
- **review / promo**: servicios de dominio nuevos que **reusan** los mismos patrones (base propia, JWT stateless, ruta en el gateway).
- **Docker** (docker-compose.full.yml): empaqueta cada servicio y lo levanta todo de una.

15. Cómo levantar lo implementado

Hay **dos formas**: jars sueltos (modo dev, una terminal por servicio) o todo en Docker (un comando).

Opción A: jars locales (modo desarrollo)

Los servicios corren como **jars**; al cerrar la terminal/sesión, se caen. Para relevantarlos:

```
docker compose up -d                                # MySQL (:3307) + RabbitMQ
(:5672, consola :15672)
cd services
# Orden: PRIMERO eureka; el resto puede ir en paralelo. Cada uno en su terminal o en
background:
java -jar eureka-server/target/eureka-server-1.0.0.jar      # :8761 (esperar a que
responda)
java -jar users-service/target/users-service-1.0.0.jar      # :8081
java -jar catalog-service/target/catalog-service-1.0.0.jar  # :8082
java -jar order-service/target/order-service-1.0.0.jar      # :8083
java -jar notification-service/target/notification-service-1.0.0.jar # :8084 (consume
eventos)
java -jar payment-service/target/payment-service-1.0.0.jar  # :8085 (cobro del checkout)
java -jar review-service/target/review-service-1.0.0.jar    # :8086 (reseñas)
java -jar promo-service/target/promo-service-1.0.0.jar      # :8087 (cupones)
java -jar api-gateway/target/api-gateway-1.0.0.jar          # :8080
```

Compilar un servicio (en Windows, pararlo antes si está corriendo — jar bloqueado): `cd services && mvn -q -pl <servicio> -am package -DskipTests` .

Opción B: todo en Docker (un solo comando)

Empaquetá los jars (`cd services && mvn -q package -DskipTests` , con los jars locales **parados** en Windows) y después:

```
docker compose -f docker-compose.full.yml up -d --build # 11 contenedores: infra + los 9 servicios
```

Un solo comando levanta MySQL (con las 6 bases ya creadas por el init SQL), RabbitMQ, Eureka, el gateway y los 7 servicios de negocio, todos hablándose por nombre de contenedor. La única puerta pública sigue siendo el gateway en `:8080` — el front no cambia.

Verificación rápida end-to-end:

```
curl http://localhost:8761/eureka/apps # ¿quién está registrado?
TOKEN=$(curl -s -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@krypton.pe","password":"Admin123!"}' | grep -o '"token":"[^"]*"' | cut
-d'' -f4)
curl http://localhost:8080/api/products # catálogo por el gateway
curl -X POST http://localhost:8080/api/admin/products \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{}' # → 400 =
token aceptado
```

Login de prueba: `admin@krypton.pe / Admin123!` .

16. Próximos pasos (se van escribiendo acá)

Con F1–F11 ya están **todos los conceptos** de la migración: descubrimiento (Eureka), gateway, JWT stateless, Feign (síncrono) + saga, máquina de estados, RabbitMQ (asíncrono), tres servicios de dominio nuevos (payment, review, promo) y el stack entero **containerizado**. El frontend ya consume los servicios nuevos, con el cupón integrado de verdad en el checkout.

Queda **una sola fase**, de pura infra y opcional:

- **F12 — Kubernetes** (*stretch*): llevar el `docker-compose.full.yml` a manifiestos de K8s (Deployments, Services, ConfigMaps/Secrets para el JWT y la DB). Es el salto de "todo en mi máquina con compose" a "orquestrado en un clúster". No agrega conceptos de dominio: es despliegue.

(Esta guía se actualiza al cerrar cada fase, siempre con el mismo nivel de detalle.)